

Table des matières

Corrigé 07 — CI/CD et qualité analytique.....	1
Questions courtes	1
Questions pratiques	4
Diagnostics	8
Questions à développer	10
Mises en situation.....	10

Corrigé 07 — CI/CD et qualité analytique

Support d'avant la refonte. La formation a été refondue le 2026-09-20, de Snowflake vers DuckDB et dbt-duckdb, sur un univers Boréal redessiné. Ce cours porte encore l'ancienne pile et l'ancienne numérotation : il sert de matière au futur Cours 08, qui n'est pas encore écrit. Les cours 01 à 06 du dossier, eux, sont à jour. Voir README.md.

Corrigé des exercices de Exercices_07.md. La numérotation est la même.

Questions courtes

1. Pour trois raisons qui valent même seul. La branche isole un chantier en cours : la principale reste toujours dans un état sain, donc reconstructible. La *pull request* est l'endroit où la CI s'accroche, sans elle, l'automatisation n'a pas de déclencheur naturel. Et l'historique devient lisible : une *pull request* raconte une intention, une suite de commits directs ne raconte rien. S'ajoute l'argument du portfolio : un dépôt avec des *pull requests* montre une façon de travailler, pas seulement un résultat.

Théorie : Theorie_07.md, section 1.

2. Une description et au moins un test sur la clé ou le grain. La syntaxe est vérifiable par une machine, c'est le rôle de `sqlfluff`, et en débattre en revue gaspille l'attention humaine sur ce qu'elle fait le moins bien. Ce que seul un relecteur peut juger, c'est si le modèle a le bon grain, s'il est réutilisable, et s'il sera compréhensible dans six mois. Un modèle sans test est une promesse sans preuve.

Théorie : Theorie_07.md, section 1.

3. Pas nécessairement. Déployer une transformation de données en production, c'est réécrire des tables que des rapports et des analystes consultent. Beaucoup d'équipes gardent une CI stricte et un déploiement déclenché volontairement, pour maîtriser le moment où la production change, par

exemple hors des heures de bureau, après le lot du soir. L'automatisation utile est celle des vérifications ; celle du déploiement est un choix d'exploitation.

Théorie : Theorie_07.md, section 2.

4. Le *runner* est une machine neuve, qui ne connaît ni votre environnement conda ni les paquets installés sur votre poste. Le workflow doit installer Python puis `dbt-snowflake` lui-même, à chaque exécution. C'est ce qui rend ces fichiers verbeux, et ce qui garantit que le résultat ne dépend pas de la machine de celui qui l'a écrit.

Théorie : Theorie_07.md, section 3.

5. Parce qu'elles n'ont ni le même coût ni le même message. `sqlfluff` prend quelques secondes, ne touche pas Snowflake et ne coûte rien ; il peut donc échouer tôt et vite. `dbt build` réveille l'entrepôt et prend des minutes. Les séparer permet d'arrêter sur une faute de forme sans avoir dépensé de crédits, et rend le journal lisible : on sait immédiatement si l'échec est de style ou de données.

Théorie : Theorie_07.md, section 4.

6. Une *pull request* cassée abîmerait la production avant même d'être relue, c'est-à-dire exactement l'inverse du but. Les rapports liraient des tables à moitié reconstruites, ou une table supprimée par un modèle renommé. La CI doit écrire dans un endroit dont personne ne dépend.

Théorie : Theorie_07.md, section 4.

7. Par la cible. La macro `generate_schema_name` du Cours 03, rendue consciente de l'environnement au Cours 05, préfixe le schéma demandé par celui de la cible active. Il suffit d'ajouter une cible `ci` avec `schema: CI` dans le `profiles.yml` de la CI : les modèles ne changent pas d'une ligne, et `+schema: marts` donne `CI_MARTS`.

Théorie : Theorie_07.md, section 4.

8. Parce que ce ne sont pas les mêmes fichiers. Celui de votre poste décrit une connexion réelle et vit hors du dépôt. Celui de la CI ne contient que des appels `env_var(...)`, des noms de variables, pas des valeurs. Il dit où chercher les identifiants, sans en porter aucun ; les valeurs vivent dans les secrets GitHub. Versionner un fichier de références est sans risque, et il faut bien que le *runner* trouve un profil quelque part.

Théorie : Theorie_07.md, section 5.

9. Non. Git conserve tout l'historique : le mot de passe est toujours lisible dans le commit précédent, et sur un dépôt public il faut considérer qu'il a été vu. Le retirer vraiment exige de réécrire l'historique, et de toute façon la seule réponse sûre est de changer le mot de passe. D'où la règle : vérifier avant le premier `git push`.

Théorie : Theorie_07.md, section 5.

10. La fraîcheur des sources. Les tests de cohérence — unicité, non-nullité, intégrité référentielle, passent parfaitement sur une table qui n’a pas bougé : elle est cohérente, elle est simplement périmée. `dbt source freshness`, avec une colonne d’horodatage de chargement et un seuil déclarés sur la source, attrape exactement ce cas.

Théorie : Theorie_07.md, section 6.

11. On règle sa sévérité et son seuil : `severity: warn`, ou un `warn_if / error_if` qui laisse passer un bruit de fond connu et alerte au-delà. Ce qu’on ne fait pas, c’est le supprimer ou l’ignorer : un test qui crie tous les jours finit par rendre invisibles ceux qui comptent, et une équipe qui prend l’habitude d’ignorer le rouge n’a plus de filet du tout.

Théorie : Theorie_07.md, section 6.

12. Parce qu’un test vérifie le contenu, un contrat verrouille la forme. Les tests disent « les valeurs de cette colonne sont uniques » ; le contrat dit « ce modèle expose ces colonnes, avec ces types ». Renommer ou retyper une colonne devient alors un échec explicite à la construction, au lieu d’une rupture découverte en aval par ceux qui consomment la table.

Théorie : Theorie_07.md, section 6.

13. Toute plateforme où la modélisation ne correspond pas à la question. Par exemple un fait au grain de la commande alors que le métier raisonne à la ligne de commande : les totaux sont cohérents, uniques, non nuls, et le chiffre d’affaires par produit est impossible à calculer. Autre cas : une marge conforme à sa définition technique mais différente de celle qu’utilise la finance. Les tests protègent la cohérence interne, jamais l’accord avec le réel.

Théorie : Theorie_07.md, section 6.

14. Parce que le `manifest.json` est un catalogue déjà écrit : modèles, colonnes, dépendances, tests, descriptions, tout y est en texte structuré, produit à chaque exécution. Sur une base ordinaire, un agent doit inférer la structure et les intentions à partir de noms de tables et d’un échantillon de données. Ici, le travail d’explicitation a déjà été fait, par les six cours précédents.

Théorie : Theorie_07.md, section 7.

15. Parce qu’il se nourrit de ce qui est écrit. Descriptions, tests, lignage, noms cohérents : sur un projet bien tenu, il a de la matière et ses propositions sont vérifiables. Sur un projet où rien n’est déclaré, il extrapole, et produit des réponses plausibles, formulées avec assurance, que personne n’est en mesure de contredire rapidement. Le risque augmente précisément là où la capacité de vérification diminue.

Théorie : Theorie_07.md, section 7.

16. Ce que fait le projet et sur quelle pile ; à quoi ressemble le résultat (une capture du lignage, une page du rapport) ; comment le rejouer. C’est le « comment le rejouer » qui manque le plus souvent, beaucoup de dépôts montrent un résultat que personne d’autre ne peut reproduire, faute de données ou d’instructions.

Théorie : Theorie_07.md, section 8.

17. Pour : le dépôt devient autonome, un visiteur peut refabriquer les données et rejouer le projet entier, ce qui est exactement ce qu'on attend d'une pièce de portfolio. Contre : le fichier existe déjà avec la formation, et un deuxième exemplaire finit par diverger du premier, on ne sait plus lequel fait foi. La façon de trancher sans regret est de nommer explicitement le maître (celui de la formation) et de traiter la copie comme un artefact d'emballage, refait à chaque publication plutôt que modifié sur place.

Théorie : Theorie_07.md, section 8.

Questions pratiques

P1.

```
name: CI dbt

on:
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    env:
      SNOWFLAKE_ACCOUNT: ${ secrets.SNOWFLAKE_ACCOUNT }
      SNOWFLAKE_USER:    ${ secrets.SNOWFLAKE_USER }
      SNOWFLAKE_PASSWORD: ${ secrets.SNOWFLAKE_PASSWORD }
      SNOWFLAKE_ROLE:    ${ secrets.SNOWFLAKE_ROLE }
      SNOWFLAKE_WAREHOUSE: ${ secrets.SNOWFLAKE_WAREHOUSE }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install dbt-snowflake sqlfluff
      - run: sqlfluff lint models/
      - run: dbt deps --profiles-dir ci
      - run: dbt build --target ci --profiles-dir ci
```

Le fichier est verbeux parce que le *runner* est une machine neuve à chaque exécution. Il ne connaît ni votre poste, ni votre environnement conda, ni votre `~/ .dbt/profiles.yml` : tout doit être installé et fourni ici. C'est le prix de la garantie que ça marchera aussi chez le prochain qui clone le dépôt.

Théorie : Theorie_07.md, sections 3 et 4.

P2.

```

boreal:
  target: ci
  outputs:
    ci:
      type: snowflake
      account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user:     "{{ env_var('SNOWFLAKE_USER') }}"
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
      role:     "{{ env_var('SNOWFLAKE_ROLE') }}"
      warehouse: "{{ env_var('SNOWFLAKE_WAREHOUSE') }}"
      database: BOREAL
      schema: CI
      threads: 4

```

Celui de votre poste contient une connexion réelle ; celui-ci ne contient que des noms de variables. Il décrit où chercher les valeurs, il ne les porte pas.

La contradiction n'est donc qu'apparente : la règle du Cours 00 n'interdit pas le fichier `profiles.yml`, elle interdit les secrets dans le dépôt. Un fichier qui n'en contient aucun peut se versionner sans risque, et il le doit même, puisque la CI en a besoin et qu'un *runner* neuf ne peut pas l'inventer.

Théorie : Theorie_07.md, sections 4 et 5.

P3.

```

[sqlfluff]
dialect = snowflake
templater = dbt
max_line_length = 100

[sqlfluff:rules:capitalisation.keywords]
capitalisation_policy = lower

```

`sqlfluff lint models/` signale, `sqlfluff fix models/` corrige la plupart des écarts.

Ce que l'outil retire des revues : tous les débats de style, majuscules des mots-clés, indentation, place des virgules, longueur de ligne. Ce sont des discussions qui coûtent du temps et de la bonne humeur sans jamais améliorer la donnée.

Ce qu'il ne retire pas, et c'est l'essentiel : les décisions de modélisation. `sqlfluff` ne dira jamais que le grain n'est pas déclaré, qu'une jointure duplique, ou qu'une règle métier est au mauvais étage. La machine prend la forme, l'humain garde le fond.

Théorie : Theorie_07.md, section 6.

P4.

```
sources:
- name: raw
  database: BOREAL
  schema: RAW
  loaded_at_field: _CHARGE_LE
  freshness:
    warn_after: {count: 36, period: hour}
    error_after: {count: 72, period: hour}
  tables:
    - name: vbap
```

La commande est `dbt source freshness`.

Ce que ça attrape et qu'aucun test de conformité ne verra : une source qui a cessé d'arriver. Un `not_null` passe parfaitement sur une table qui n'a pas bougé depuis trois semaines, puisque les lignes présentes sont toujours aussi valides. Les tests répondent à « la donnée est-elle conforme », la fraîcheur répond à « la donnée est-elle encore là ». C'est souvent le contrôle le plus utile d'un projet, et presque toujours le dernier écrit.

Théorie : Theorie_07.md, section 6.

P5.

```
- name: code_postal
  tests:
    - dbt_utils.expression_is_true:
        expression: "regexp_like(code_postal, '^[A-Z][0-9][A-Z] ?[0-9][A-Z][0-9]$')"
        config:
          severity: warn
          warn_if: ">5"
          error_if: ">50"
```

Un test qui ne doit jamais être gradué : l'unicité d'une clé de dimension, ou l'intégrité référentielle d'un fait.

```
- name: concess_key
  tests: [unique, not_null]
```

La raison est de nature, pas de degré. Un code postal mal formaté est une imperfection de saisie qui ne fausse aucun total : le rapport reste juste. Une clé en double fausse mécaniquement toutes les sommes qui passent par cette dimension. Grader le second reviendrait à décider que des chiffres faux peuvent atteindre la production sans arrêter personne, et l'expérience dit que le seuil n'est jamais remonté ensuite.

Théorie : Theorie_07.md, section 6.

P6.

```
models:
- name: fct_ventes
  config:
    contract:
      enforced: true
  columns:
    - name: date_key
      data_type: number(8,0)
      constraints: [{type: not_null}]
    - name: concess_key
      data_type: varchar
      constraints: [{type: not_null}]
    - name: produit_key
      data_type: varchar
      constraints: [{type: not_null}]
    - name: commande_id
      data_type: varchar
    - name: poste_no
      data_type: varchar
    - name: quantite
      data_type: number(12,3)
    - name: montant_vente
      data_type: number(18,2)
    - name: marge
      data_type: number(18,2)
```

Désormais, renommer une colonne fait échouer la construction du modèle, avec un message qui dit exactement quelle colonne du contrat manque. Avant, la construction réussissait : le modèle produisait simplement une table différente, et la rupture se manifestait plus loin, dans un rapport Power BI ou un script d'export, souvent des jours plus tard et sans lien apparent avec le changement.

La différence tient en un mot : le contrat transforme une surprise en aval en une rupture explicite à la source. C'est le pendant, côté données, de la revue de code.

Théorie : Theorie_07.md, section 6.

P7. Cinq points au moins, et le premier est le seul qui empêche vraiment le code de fonctionner correctement dans la durée.

Bloquants :

1. Les noms de table sont en dur. Il faut `ref('stg_lfa1')` et `ref('stg_t005')`. Sans ça, pas de graphe, donc pas d'ordre d'exécution garanti, pas de lignage, et `--select inopérant`.

2. Aucune clé de substitution. La dimension expose la clé métier comme identifiant, ce qui interdit tout passage ultérieur en SCD type 2 et oblige les faits à joindre pour retrouver leur clé.
3. Aucun fichier `.yaml`. Il manque la description du modèle et de ses colonnes, et surtout les tests, au minimum `unique` et `not_null` sur la clé. Un modèle sans test est une promesse sans preuve.

Remarques, non bloquantes :

4. Le `LEFT JOIN` est le bon choix, il mérite d'être signalé comme tel plutôt que corrigé : un fournisseur sans pays connu doit rester dans la dimension. Une remarque en revue confirme que c'était délibéré.
5. Le nom du fichier annonce un mart, mais le modèle se contente d'aplatir deux stagings. C'est acceptable pour une dimension, il faut simplement vérifier que la convention du projet le prévoit et que le dossier est le bon.

Théorie : Theorie_07.md, section 1.

P8. Dans l'ordre, et l'urgence n'est pas le nettoyage.

Urgent, dans l'heure :

1. Changer le mot de passe Snowflake, et le changer partout où il a pu être réutilisé.
2. Vérifier les journaux d'accès du compte Snowflake pour la période d'exposition.

Nettoyage, ensuite :

3. Réécrire l'historique pour retirer le fichier de tous les commits, avec `git filter-repo` ou l'outil équivalent, puis forcer la mise à jour du dépôt distant.
4. Ajouter `profiles.yaml` au `.gitignore` et déplacer le fichier vers `~/dbt/`.
5. Considérer malgré tout le secret comme brûlé : sur un dépôt public, il a pu être moissonné dans les minutes suivant le premier push, et les forks éventuels conservent l'ancien historique.

```
dbt build --select state:modified+ --state ./artefacts_prod --target ci
```

La CI mince exige en retour qu'on conserve et qu'on publie les artefacts de la dernière exécution de production, notamment le `manifest.json`, dans un endroit que la CI peut lire. C'est ce `--state` qui permet à dbt de comparer et de déduire ce qui a changé. Sans cette mécanique de conservation, le sélecteur ne peut rien comparer et reconstruit tout.

Théorie : Theorie_07.md, section 5.

Diagnosics

E1. La CI construisait dans les schémas de production.

À chaque proposition de changement, le workflow lançait `dbt build --target prod`, donc écrivait dans `MARTS` et `STAGING`. Ce matin-là, une proposition contenait un modèle incomplet ou un test rouge, la construction s'est arrêtée en cours de route, et la production est restée dans l'état où la CI l'avait laissée. Aucun changement n'avait été fusionné, et pourtant la production était cassée : c'est exactement ce que la CI est censée empêcher.

Personne ne l'avait vu parce que, jusque-là, toutes les propositions étaient correctes. La CI reconstruisait la production à l'identique, ce qui ne se remarque pas. Le défaut n'apparaît qu'au premier échec, c'est-à-dire au premier moment où la CI sert enfin à quelque chose.

Le correctif ne tient pas en un mot changé, et c'est le point de l'exercice. Il faut : une cible `ci` dans le profil versionné ; une branche dans `generate_schema_name` qui préfixe en `CI_` pour cette cible ; des droits Snowflake qui autorisent la CI à créer ses schémas et lui interdisent d'écrire dans ceux de production ; et enfin le `--target ci` dans le workflow. Le dernier point seul laisserait la CI capable d'écrire en production le jour d'une faute de frappe : la vraie protection est dans les droits, pas dans le paramètre.

Théorie : Theorie_07.md, section 2.

E2. Git garde tout. Le fichier n'apparaît plus dans l'état actuel de la branche, mais il est intégralement présent dans le commit où il a été ajouté, que n'importe qui peut consulter avec `git log -p` ou en naviguant dans l'historique sur GitHub. Sur un dépôt public, il est en outre indexé et moissonné par des robots qui scrutent précisément ce genre de fichier, souvent dans les minutes qui suivent le push.

Ce qui est exposé : le mot de passe, mais aussi le nom du compte Snowflake, l'utilisateur, le rôle et l'entrepôt, c'est-à-dire tout ce qu'il faut pour se connecter. Et si ce mot de passe est réutilisé ailleurs, l'exposition dépasse largement ce projet.

Ce qu'il faut faire : changer le mot de passe d'abord, réécrire l'historique ensuite, et considérer le secret comme brûlé quoi qu'il arrive, parce que les copies et les forks éventuels ne se réécrivent pas.

Ce qui aurait dû l'empêcher : le `.gitignore` posé au Cours 00, et le réflexe de ne jamais mettre le fichier dans le projet. On peut ajouter un crochet de pré-commit qui refuse un fichier contenant un motif de mot de passe. La vérification après coup arrive toujours trop tard parce que le push est irréversible : ce n'est pas une opération qu'on annule, c'est une publication.

Théorie : Theorie_07.md, section 5.

E3. L'enchaînement est le plus banal et le plus coûteux qui soit.

Trente tests bruyants ont créé une fatigue d'alerte : un build rouge en permanence ne dit plus rien, donc personne ne le regarde. Pour retrouver un signal, l'équipe a désactivé le bruit, mais elle l'a fait globalement, en passant tous les tests en avertissement. Elle a donc éteint en même temps les trente tests inutiles et les cent dix qui protégeaient vraiment. Six semaines plus tard, le test d'unicité qui aurait attrapé le doublon a émis un avertissement dans un journal que personne ne lit.

Ce qu'il fallait faire des trente : les traiter un par un, ce qui demande une journée et pas cinq minutes. Pour chacun, trois issues possibles. Corriger la donnée à la source, quand l'anomalie est réelle et réparable. Graduer le test avec un seuil, quand l'imperfection est structurelle et sans conséquence sur les chiffres. Ou

supprimer le test, quand il vérifiait une règle qui n’a jamais été vraie et n’aurait pas dû être écrite. Un test qu’on garde en rouge permanent est le pire des trois choix.

Le principe qui distingue : on gradue un test qui mesure une imperfection sans conséquence sur les totaux, on bloque avec un test qui garantit une propriété structurelle du modèle, unicité d’une clé, intégrité référentielle, respect du grain. Le critère n’est pas le nombre de lignes fautives, c’est de savoir si un chiffre publié devient faux.

Théorie : Theorie_07.md, section 6.

Questions à développer

A. Le mécanisme est presque toujours le même, et il ne commence pas par de la négligence. Au départ, la CI est lente : elle reconstruit tout le projet à chaque *pull request*, et l’attente devient un péage. Ensuite, elle est bruyante : des tests trop stricts ou mal calibrés échouent sur des données réelles imparfaites, si bien que le rouge devient normal. Une fois que le rouge est normal, il ne signifie plus rien, et la première échéance serrée fournit la justification de la première fusion forcée. La suite est mécanique : ce qu’on contourne une fois par exception, on le contourne ensuite par habitude, et le workflow finit désactivé « temporairement ». Autrement dit, une CI ne meurt pas d’être trop faible, elle meurt d’être trop coûteuse et trop bavarde. Une CI conçue pour survivre applique donc trois principes. Elle est rapide : on échoue d’abord sur la forme, qui coûte quelques secondes, et on ne reconstruit que ce qui a changé plutôt que le projet entier. Elle est calibrée : chaque test a une sévérité et un seuil réfléchis, de sorte qu’un rouge signifie toujours « il faut agir maintenant », le bruit connu est en avertissement, pas en échec. Elle est enfin honnête sur son périmètre : elle vérifie ce qu’elle sait vérifier, et l’on ne prétend pas qu’elle remplace la revue humaine sur le grain et la modélisation. Le vrai indicateur de santé n’est pas le nombre de tests, c’est la proportion d’exécutions rouges qui débouchent sur une correction.

Théorie : Theorie_07.md, section 2.

B. Le plan qui fonctionne va du problème au résultat, puis à la méthode, jamais l’inverse. On ouvre sur la situation de départ en une phrase : des extraits d’ERP illisibles, noms techniques, dates en texte, aucune analyse possible sans y passer la journée. On montre ensuite l’arrivée, parce que c’est ce qui donne envie d’écouter la suite : une page du rapport Power BI, marge par région et par gamme, et le fait qu’elle se recalcule seule après chaque lot mensuel. On remonte alors la chaîne, brièvement : le brut entre une fois, le staging le rend lisible table par table, l’étoile l’assemble au grain de la ligne de commande, les tests la protègent. Les chiffres à citer sont ceux qui prouvent une propriété, pas ceux qui impressionnent : le nombre de lignes du fait égal à celui de la source, le grain est préservé ; le nombre de tests et le fait que la CI les rejoue à chaque *pull request* ; le temps de reconstruction. On termine par une décision de conception qu’on peut défendre, par exemple pourquoi la marge est calculée dans dbt et non en DAX. La faiblesse à assumer d’avance doit être réelle et choisie : ici, que les données sont générées et non issues d’un vrai ERP, donc que le projet démontre une méthode et non une performance métier ; ou que le déploiement en production reste manuel. Annoncer soi-même la limite déplace la conversation de « qu’est-ce qui manque ? » vers « pourquoi ce choix ? », et c’est là qu’on est bon.

Théorie : Theorie_07.md, section 8.

Mises en situation

M1. Imprimerie Sorel.

Le dispositif, à chaque proposition de changement, dans cet ordre :

Étape	Bloquant	Pourquoi
sqlfluff lint	oui	rapide, gratuit, ne touche pas l'entrepôt ; échoue en 20 secondes plutôt qu'en 22 minutes
dbt deps	oui	sans les paquets, rien ne compile
dbt build sur les modèles touchés	oui	c'est le vrai filet : le projet se construit-il et les tests passent-ils
dbt source freshness	non	une source en retard n'est pas la faute de la proposition

La CI écrit dans ses propres schémas, préfixés `CI_`, jamais ailleurs. La raison est nette : si elle construisait dans MARTS, une proposition cassée abîmerait la production avant même d'être relue, c'est-à-dire exactement ce qu'on cherche à empêcher. Et la protection se pose au niveau des droits Snowflake, pas seulement du paramètre `--target`, parce qu'un paramètre s'oublie.

Sur le coût, l'option est la CI mince. En comparant l'état de la proposition à celui de la dernière exécution de production, dbt ne reconstruit que ce qui a changé et ce qui en dépend, avec `--select state:modified+`. Sur des propositions qui touchent un ou deux modèles parmi 180, on passe de 22 minutes à moins d'une. Ce qu'il faut mettre en place avant : conserver et publier les artefacts de la production, en particulier le `manifest.json`, dans un emplacement que la CI peut lire. Sans cet état de référence, il n'y a rien à comparer.

Sur le déploiement continu : non, pas ici, et c'est une position défendable plutôt qu'une prudence. Sur un projet analytique, la production se reconstruit selon un cycle lié à l'arrivée des données, pas au moment où une revue est approuvée. Déclencher une reconstruction complète à chaque fusion produirait des exécutions à contretemps, des coûts imprévisibles, et un rapport qui change pendant qu'on le regarde. Une CI stricte et un déploiement déclenché volontairement est le bon réglage pour la plupart des équipes analytiques.

Théorie : Theorie_07.md, sections 2 et 4.

M2. Éoliennes Matane. La revue :

Bloquants :

1. `stg_arret` joint deux tables. Un modèle de staging est 1 pour 1 avec sa source et ne joint rien. Il faut deux modèles de staging, et la jointure va dans un intermédiaire ou dans le mart.

2. Aucun fichier `.ym1`. Pas de description, et surtout pas de test. Il manque au minimum le test qui prouve le grain de `fct_arret`, qui est ici le couple turbine et jour.
3. Le grain de `fct_arret` n'est pas déclaré. Le modèle agrège par turbine et par jour, mais rien ne le dit ni ne le vérifie, et c'est l'erreur la plus coûteuse et la plus silencieuse.

Non bloquant :

4. Le message de commit « ajout arrêts » ne raconte rien. À demander, sans bloquer la fusion pour ça.

Les deux points qui ne sont pas des fautes de SQL mais des décisions de modélisation à expliciter sont le grain de `fct_arret` et le filtre sur le motif. Le premier parce qu'agréger au jour est un choix qui ferme des questions : on ne pourra plus analyser la durée d'un arrêt individuel ni les arrêts qui chevauchent minuit. Le second parce qu'il retire de la donnée sans le dire.

La question à poser sur le filtre, et c'est la plus importante de la revue : pourquoi les maintenances planifiées sont-elles exclues, et qui a pris cette décision ? Ce qui inquiète est double. D'abord, un filtre métier non commenté devient invisible en trois mois, et le prochain lecteur croira que le fait contient tous les arrêts. Ensuite, ce choix change la définition de la disponibilité d'une turbine, qui est probablement l'indicateur le plus regardé de l'entreprise : selon qu'on inclut ou non les arrêts planifiés, le chiffre publié n'est pas le même, et il doit correspondre à ce que le métier appelle disponibilité. C'est typiquement une question à poser à quelqu'un des opérations, pas à trancher en revue de code.

Théorie : `Theorie_07.md`, section 1.

M3. Coopérative Saint-Lambert.

Janvier, le fichier qui n'arrive plus. Le dispositif est `dbt source freshness`, avec un `loaded_at_field` déclaré sur la source et des seuils d'avertissement et d'erreur. Les 90 tests ne pouvaient rien y faire : ils vérifient la conformité des lignes présentes, et les lignes présentes étaient parfaitement valides. Aucun test de conformité ne détecte une absence.

Mars, la colonne renommée. Le dispositif est le contrat de modèle, qui déclare les colonnes et leurs types et fait échouer la construction en cas de rupture. Les tests ne pouvaient rien y faire non plus : le modèle produisait une table valide, simplement différente. La rupture s'est manifestée chez les consommateurs, en aval, sans lien apparent avec le changement. On peut ajouter les `exposure` déclarées dans `dbt`, qui auraient au moins montré, avant la fusion, que deux rapports dépendaient de cette colonne.

Juin, la marge qui n'est pas la marge. C'est celui qui n'a aucune solution technique. Aucun test ne dira jamais si la définition que vous avez codée est celle que la finance emploie : les tests protègent la cohérence, pas la pertinence. Ce qui le prévient réellement est une conversation, et un processus, faire valider chaque définition d'indicateur par le service qui en est propriétaire, l'écrire dans la `description` du modèle, et nommer la mesure de façon à ce qu'un écart de définition se voie, par exemple `marge_brute_hors_remeses` plutôt que `marge`.

Par ordre de gravité, du plus grave au moins grave : juin, janvier, mars. Le critère est la durée pendant laquelle une décision a pu être prise sur un chiffre faux sans que personne le sache. Juin a duré depuis la mise en service, et c'est le pire des trois parce que tout le monde faisait confiance. Janvier a duré onze jours,

avec un chiffre incomplet mais non faux en soi. Mars a été bruyant et détecté tout de suite, deux rapports cassés se signalent d'eux-mêmes ; c'est le plus visible et le moins dangereux.

Théorie : Theorie_07.md, section 6.

M4. (avancé) Gestion Portneuf.

Ce qui rend une plateforme dbt hospitalière à un agent, c'est qu'elle est déjà entièrement écrite. Chaque exécution produit un `manifest.json`, le catalogue complet du projet, les modèles, les colonnes, les dépendances, les tests, les descriptions. Un agent n'a pas à deviner la structure de l'entrepôt, elle lui est servie en texte structuré. C'est une différence de nature avec une base sans couche de transformation, où tout devrait être inféré à partir des noms.

Ce qui manque ici, dans l'ordre où le combler :

1. Les définitions des trois mesures clés. Une seule définition par indicateur, dans dbt, validée par le service propriétaire. C'est premier parce que c'est le seul point dont dépendent tous les autres : un agent qui répond juste selon une définition et faux selon l'autre est inutilisable.
2. Les grains, déclarés dans les `description` et prouvés par des tests. Deux personnes qui savent ne suffisent pas, et l'agent ne lit pas dans les têtes.
3. Les descriptions manquantes, sur la moitié des modèles. C'est le matériau que l'agent lit, et c'est aussi le travail où il peut aider, en proposant des ébauches qu'un humain valide.
4. Les tests au-delà des `not_null` : unicité des clés, intégrité référentielle, fraîcheur des sources. Ils ne servent pas l'agent directement, ils servent la confiance dans ce qu'il répondra.

Ce que l'agent pourra faire de crédible ensuite : proposer les descriptions manquantes, signaler en revue un `ref()` absent ou un modèle sans test, et traduire une question métier en requête sur les marts en s'appuyant sur le lignage. Ce qui reste humain : valider une description, car une description inventée est pire qu'absente ; décider si le modèle est le bon ; et vérifier un chiffre avant de le diffuser.

Si on branchait l'agent aujourd'hui, le mode d'échec n'est pas celui qu'on imagine. On craint qu'il refuse de répondre ou qu'il se trompe visiblement. Ce qui arrivera est l'inverse : il répondra à tout, avec assurance, en choisissant une des trois définitions du taux d'occupation sans dire laquelle, et personne ne saura que la réponse dépendait d'un arbitrage. Une plateforme mal tenue ne produit pas un agent muet, elle produit un agent plausible et faux, ce qui est bien plus coûteux qu'un outil qui échoue franchement.

La phrase de trente secondes : un agent lit la plateforme, il ne la répare pas ; il est d'autant plus utile que le grain, les définitions et les tests sont déjà en place, donc le travail à faire avant l'IA est exactement celui qu'on devait faire de toute façon.

Théorie : Theorie_07.md, section 7.